Solution to Divide and Conquer - countZeroes

```
public class Exercise {
  public static int countZeroes(int[] array) {
   int left = 0;
   int right = array.length;
   while (left <= right) {
     int middle = (int) (left+right)/2;
     if (array[middle]==1) {
       left = middle + 1;
     } else {
       right = middle - 1;
     }
   }
   return array.length - left;
 }


}
```

Solution to Divide and Conquer - sortedFrequency

```
public class Exercise {

public static int sortedFrequency(int[] arr,
int num) {
   int left = 0;
   int right = arr.length - 1;
   int leftCount = 0;
   int rightCount = 0;
   while (left <= right) {
     int middle = (int) Math.floor((left +
right) / 2);
     if (arr[middle] == num) {
       leftCount = middle;
```

```
        rightCount = middle;
        while (arr[leftCount]==num && leftCount
>= 0) {
          leftCount -= 1;
        }

        while (rightCount<arr.length &&
arr[rightCount]  == num) {
          rightCount += 1;
        }
        return rightCount - leftCount - 1;
      }
      if (arr[middle] < num) {
        left = middle + 1;
      } else {
        right = middle - 1;
      }
    }
    return -1;
  }

}
```

Solution to Divide and Conquer - findRotatedIndex

```
public class Exercise {

public static int findRotatedIndex(int[] arr,
int num) {
    int left = 0;
    int right = arr.length - 1;
    int middle = 0;
    if (right>0 && arr[left] >= arr[right]) {
      middle = (int) Math.floor((left + right)
/ 2);
```

```
      while (arr[middle] <= arr[middle + 1]) {
        if (arr[left] <= arr[middle]) {
          left = middle + 1;
        } else {
          right = middle - 1;
        }
        middle = (int) Math.floor((left +
right) / 2);
        if (middle+1 > arr.length-1) {
          break;
        }

      }
      if (num >= arr[0] && num <= arr[middle])
{
        left = 0;
        right = middle;
      } else {
        left = middle + 1;
        right = arr.length - 1;
      }

    }
    while (left <= right) {
      middle = (int) Math.floor((left + right)
/ 2);
      if (num == arr[middle]) {
        return middle;
      }
      if (num > arr[middle]) {
        left = middle + 1;
      } else {
        right = middle - 1;
      }
    }
```

```
        return -1;
    }

}
```